Priority Queues: Binary Heaps

Last Class

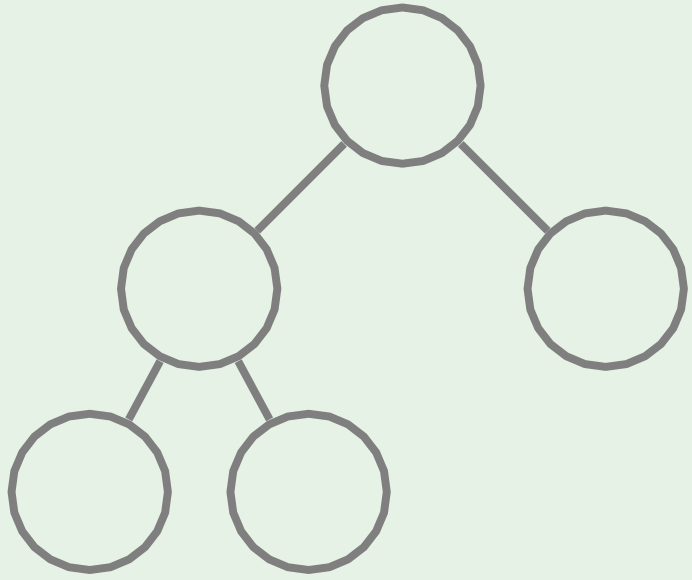
- `GetMax` works in time $O(1)$, all other operations work in time O(tree height)
- we definitely want a tree to be shallow

How to Keep a Tree Shallow?

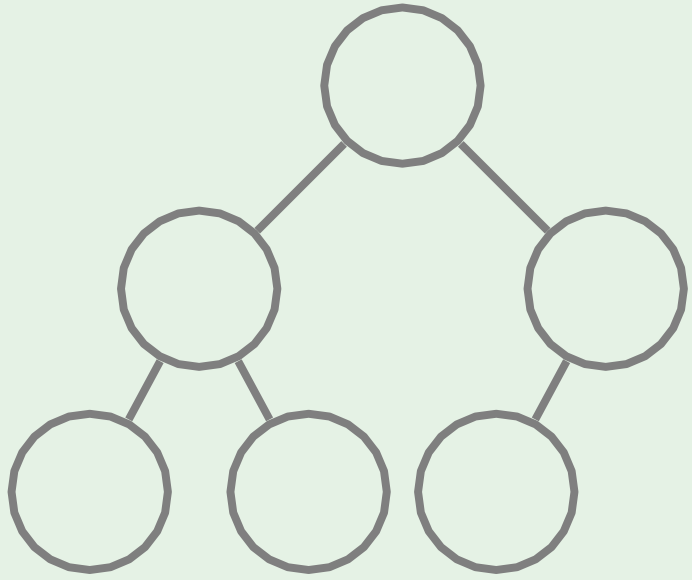
Definition

A binary tree is complete if all its levels are filled except possibly the last one which is filled from left to right.

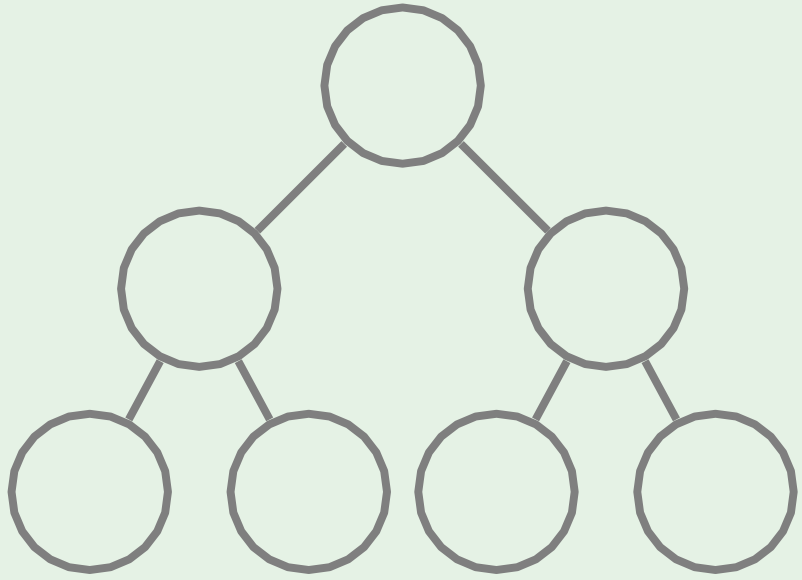
Example: complete binary tree



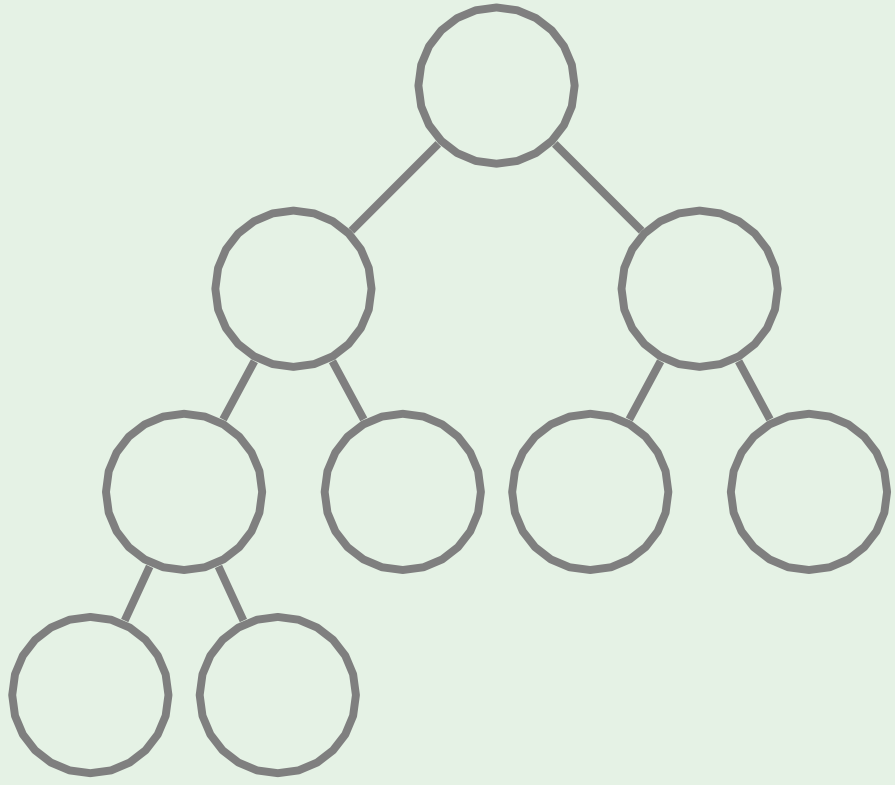
Example: complete binary tree



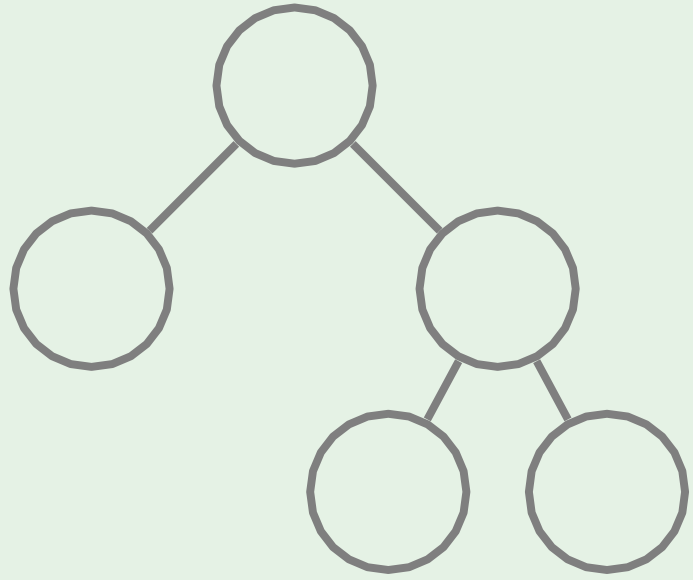
Example: complete binary tree



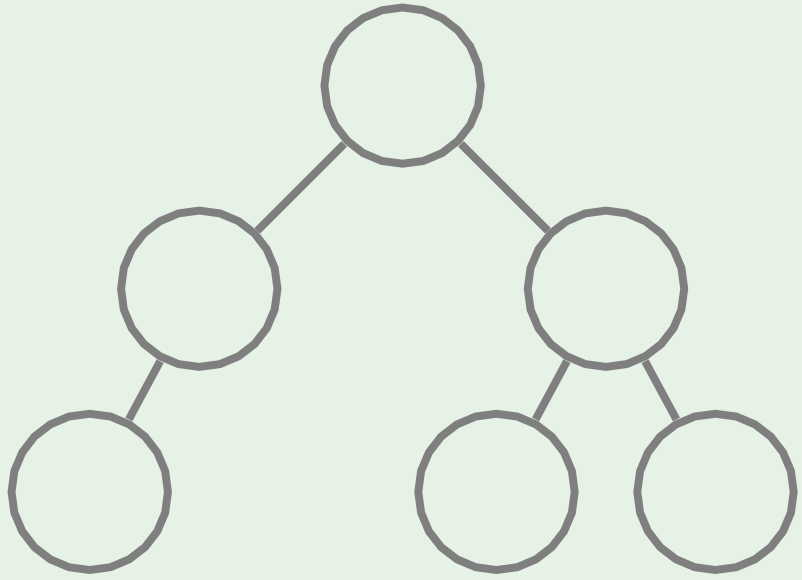
Example: complete binary tree



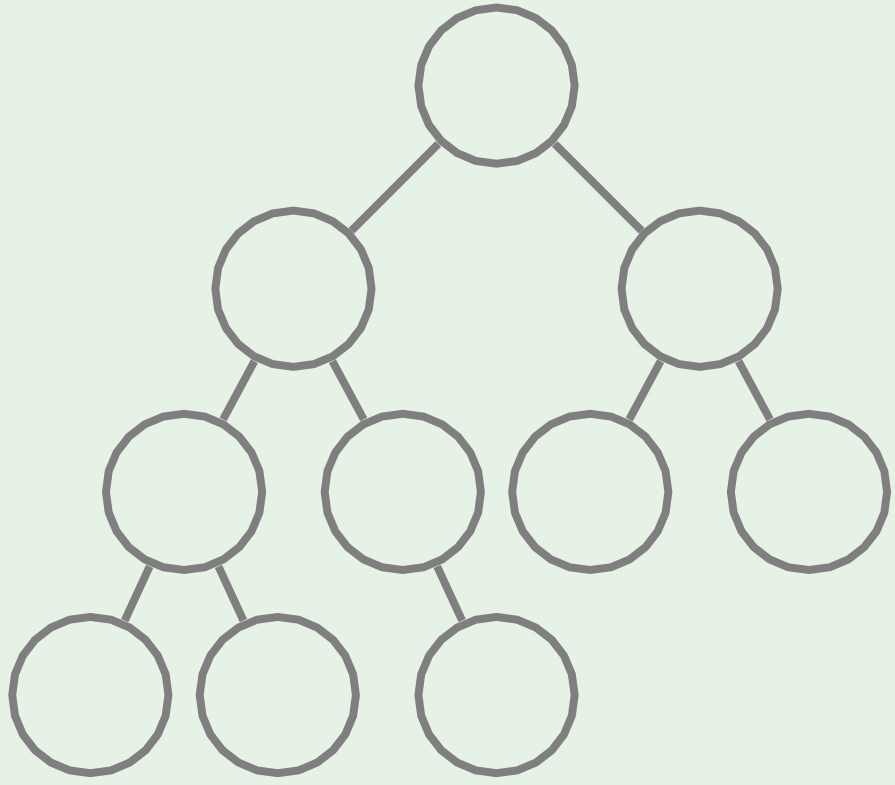
Example: not complete binary tree



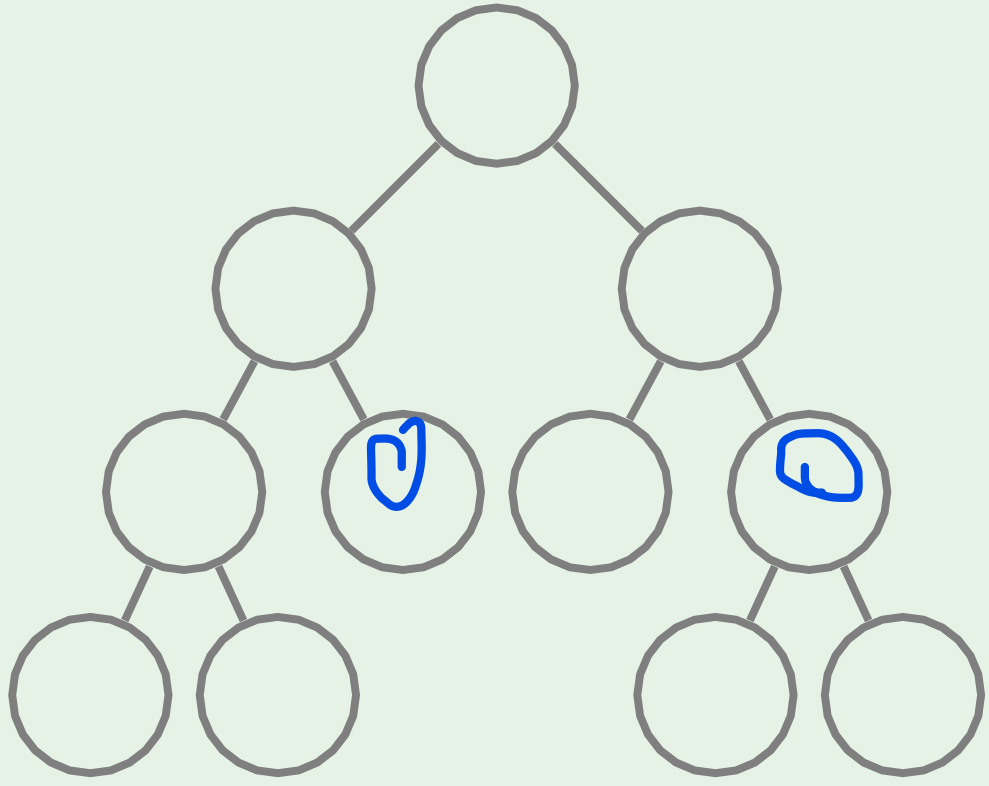
Example: not complete binary tree



Example: not complete binary tree



Example: not complete binary tree



First Advantage: Low Height

Lemma

A complete binary tree with n nodes has height at most $O(\log n)$.

Proof

In a complete binary tree:

- The k-th level can hold at most 2^k nodes.
- The maximum number of nodes up to a height h (if all levels are fully filled) is given by the sum of a geometric series:

$$1 + 2 + 4 + \cdots + 2^h = 2^{h+1} - 1$$

- So, a complete binary tree with height h has at most $2^{h+1} - 1$ nodes.

Let n be the total number of nodes in the complete binary tree. Then:

$$n \leq 2^{h+1} - 1$$

Solving for h, we get:

$$2^{h+1} \leq n + 1$$

Taking the logarithm on both sides:

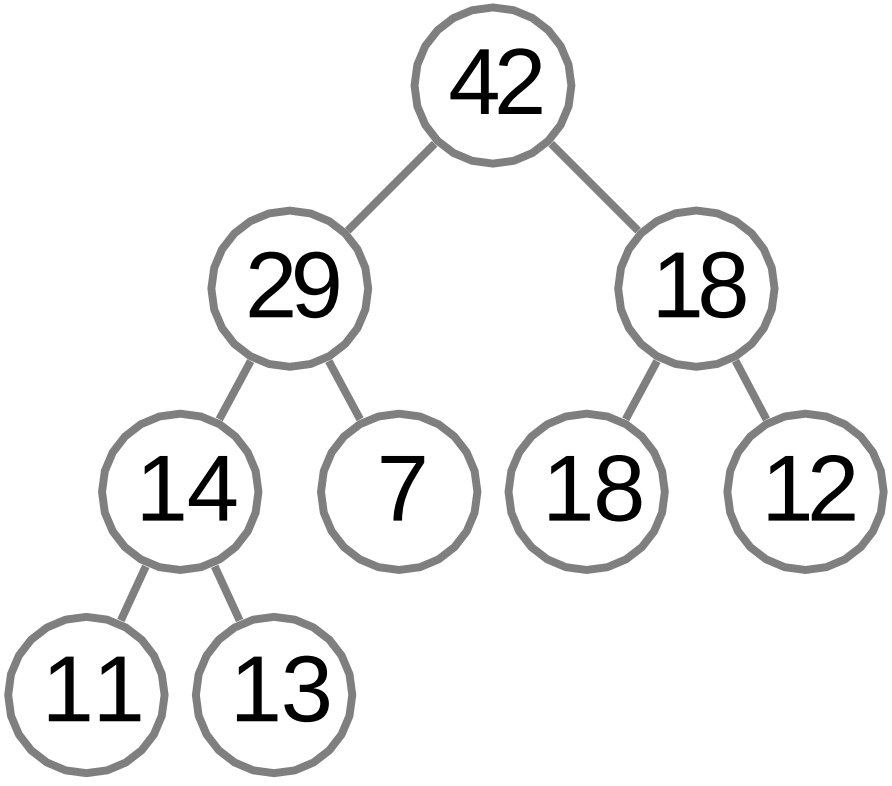
$$h + 1 \leq \log_2(n + 1)$$

So:

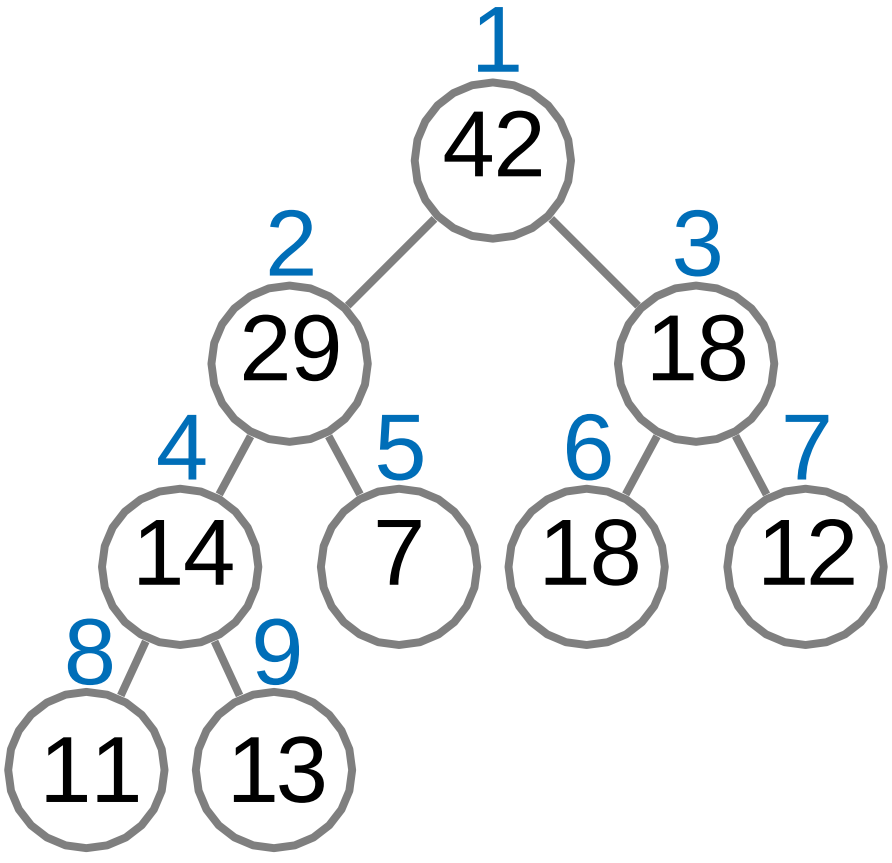
$$h \leq \log_2(n + 1) - 1$$

This shows that the height h is at most $O(\log n)$.

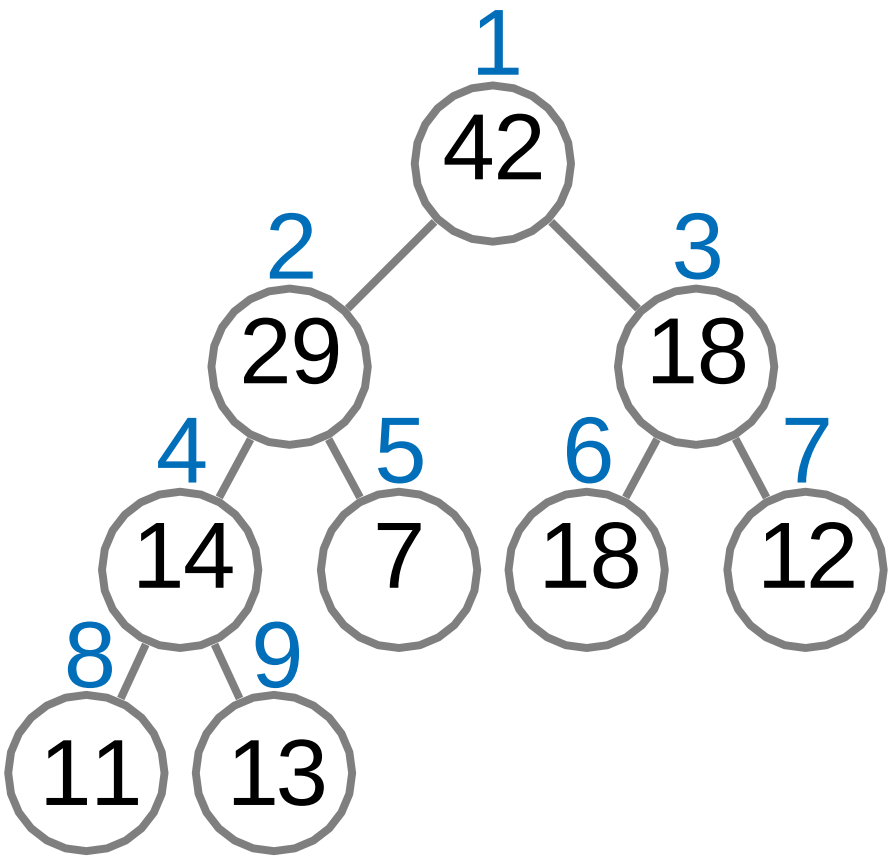
Second Advantage: Store as Array



Second Advantage: Store as Array



Second Advantage: Store as Array

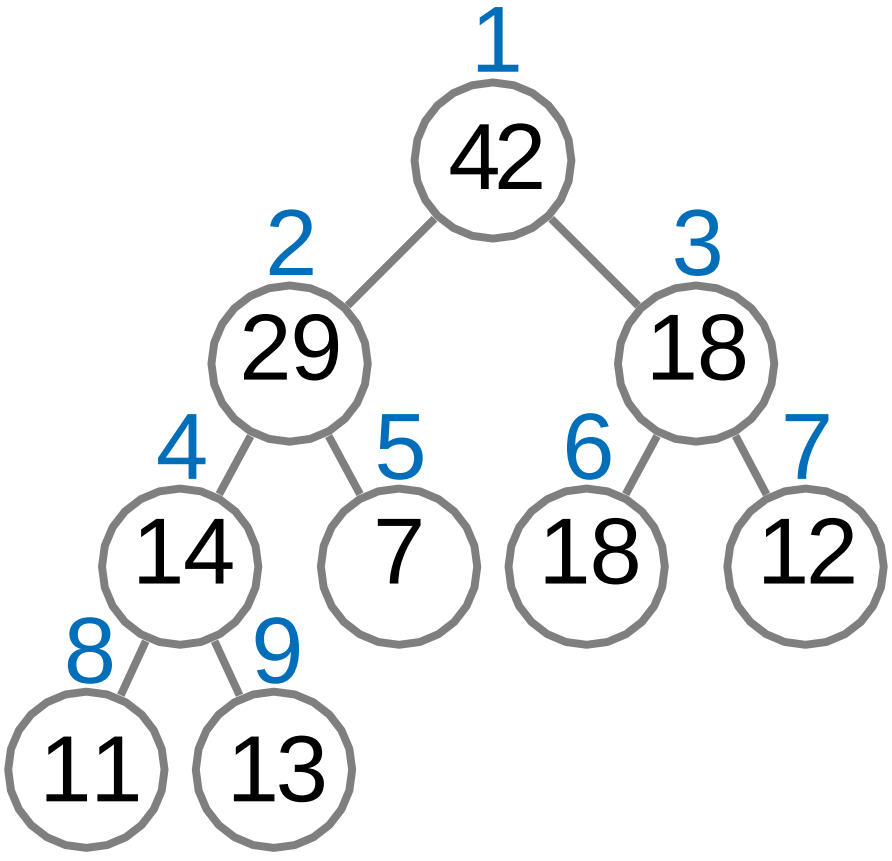


parent(i) = $\lfloor i/2 \rfloor$ floor of i/2

leftchild(i) = $2i$

rightchild(i) = $2i + 1$

Second Advantage: Store as Array



parent(i) = $\lfloor i/2 \rfloor$

leftchild(i) = $2i$

rightchild(i) = $2i + 1$

1	2	3	4	5	6	7	8	9
42	29	18	14	7	18	12	11	13

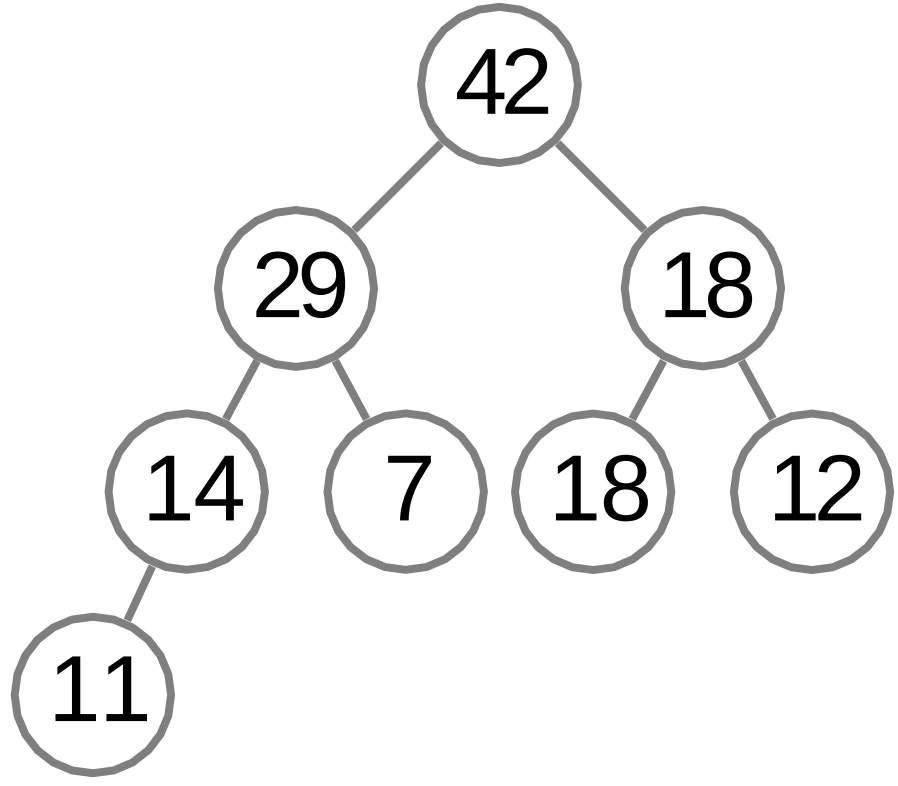
- What do we pay for these advantages?

- What do we pay for these advantages?
- We need to keep the tree complete.

- What do we pay for these advantages?
- We need to keep the tree complete.
- Which binary heap operations modify the shape of the tree?

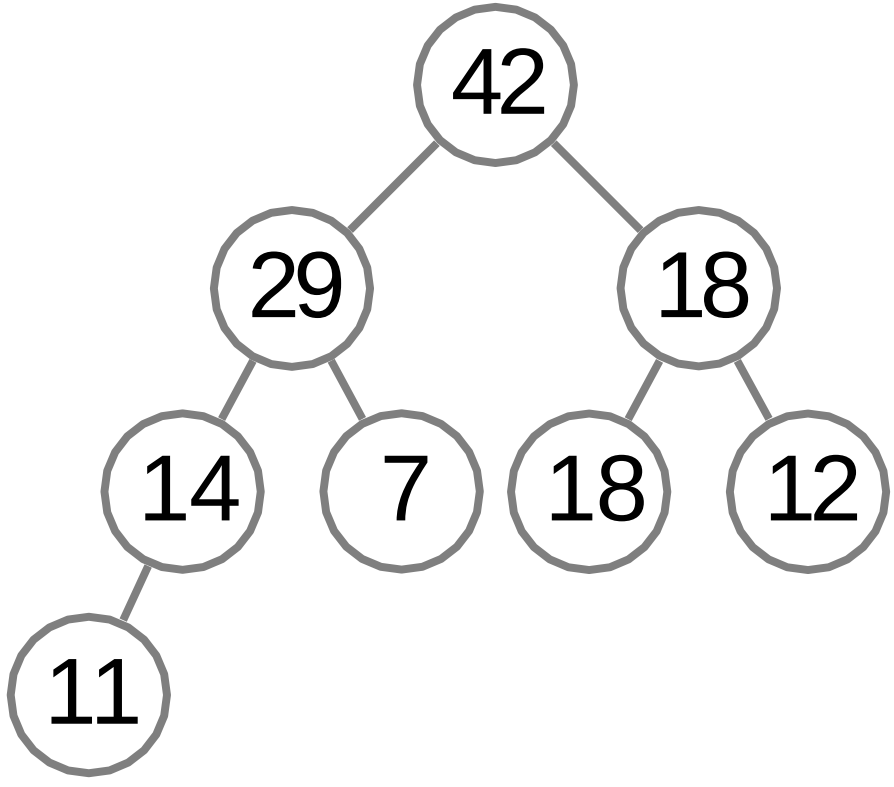
- What do we pay for these advantages?
- We need to keep the tree complete.
- Which binary heap operations modify the shape of the tree?
- Only `Insert` and `ExtractMax` (`Remove` changes the shape by calling `ExtractMax`).

Keeping the Tree Complete



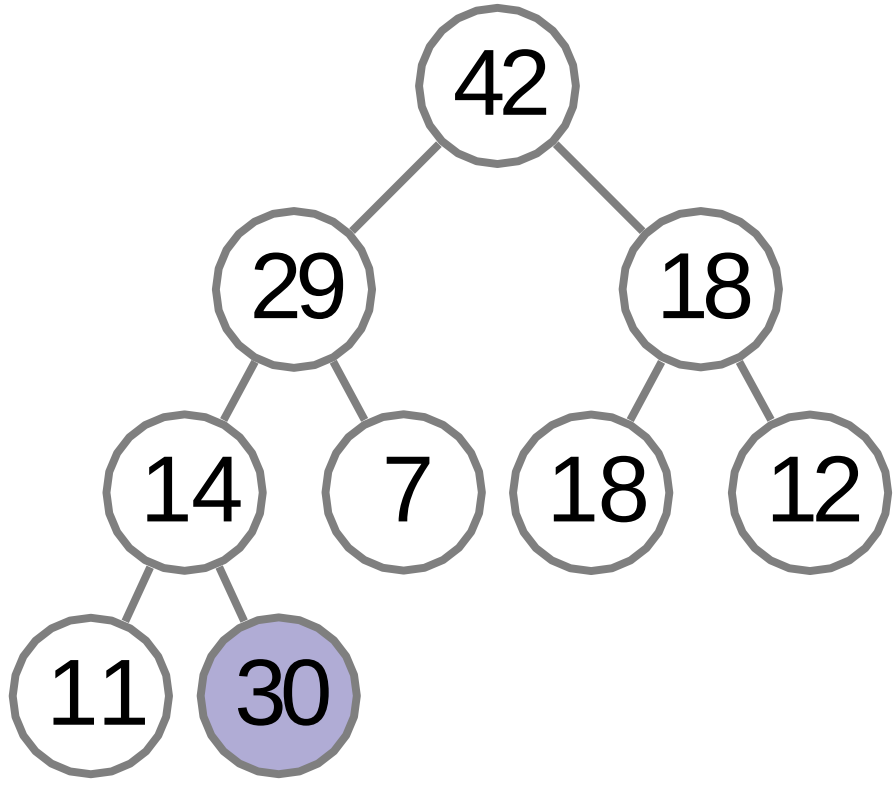
Keeping the Tree Complete

to insert an element, insert it as a leaf in the leftmost vacant position in the last level and let it sift up



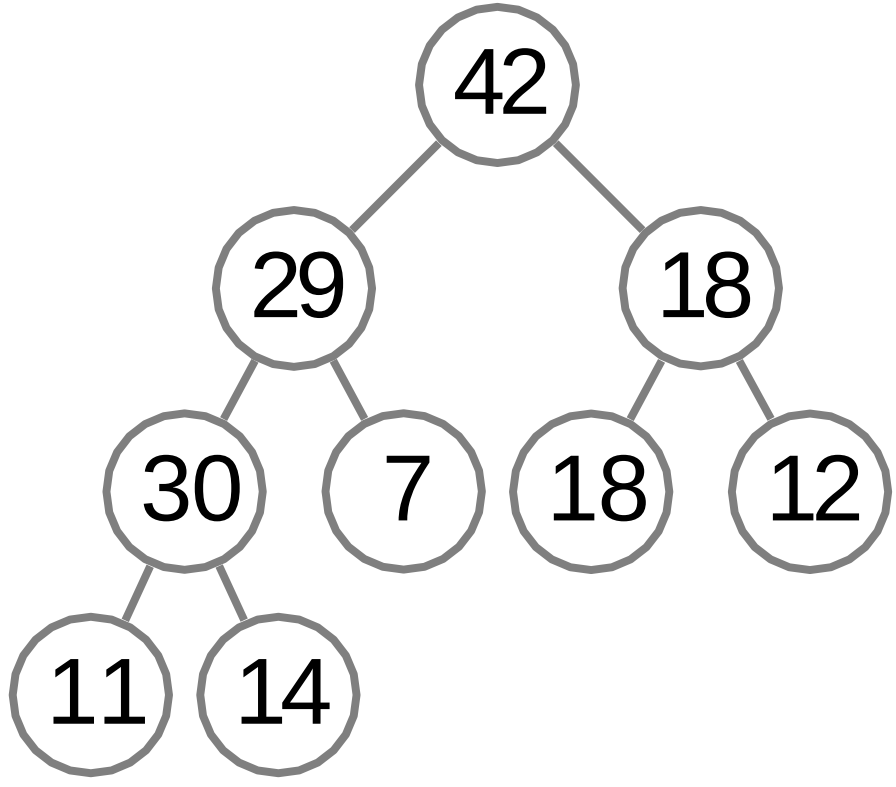
Keeping the Tree Complete

to insert an element, insert it as a leaf in the leftmost vacant position in the last level and let it sift up



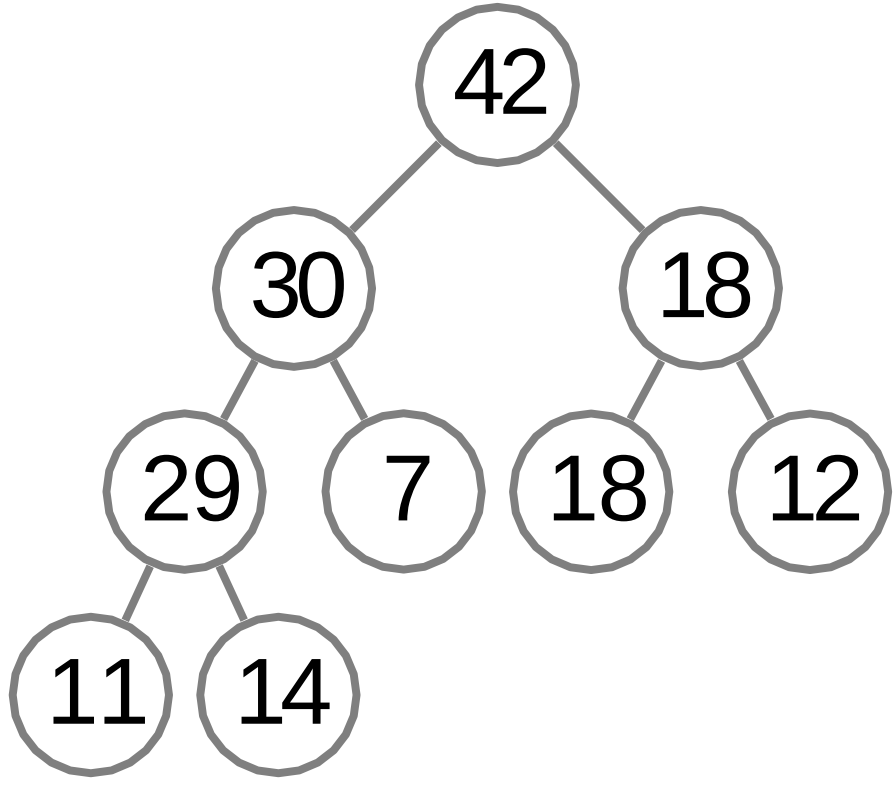
Keeping the Tree Complete

to insert an element, insert it as a leaf in the leftmost vacant position in the last level and let it sift up



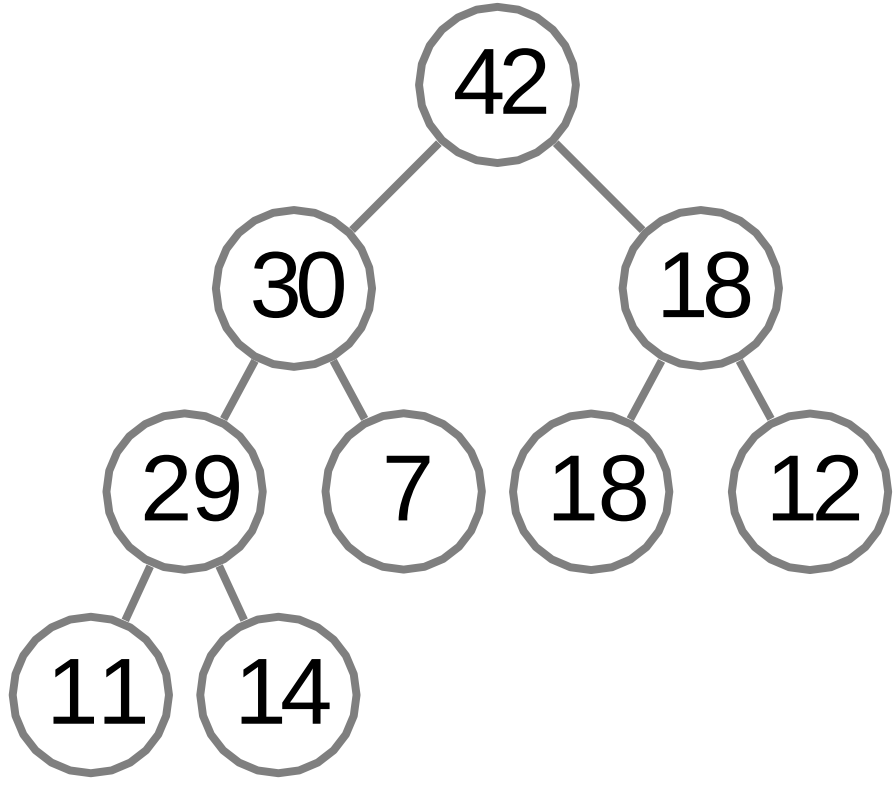
Keeping the Tree Complete

to insert an element, insert it as a leaf in the leftmost vacant position in the last level and let it sift up



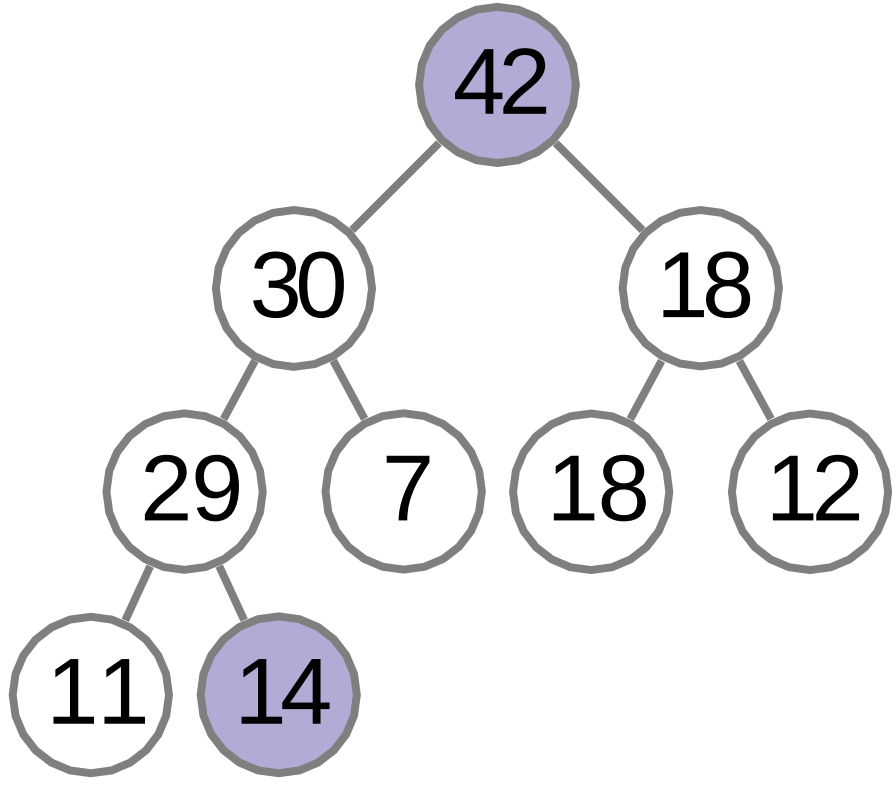
Keeping the Tree Complete

to extract the maximum value, replace the root by the last leaf and let it sift down



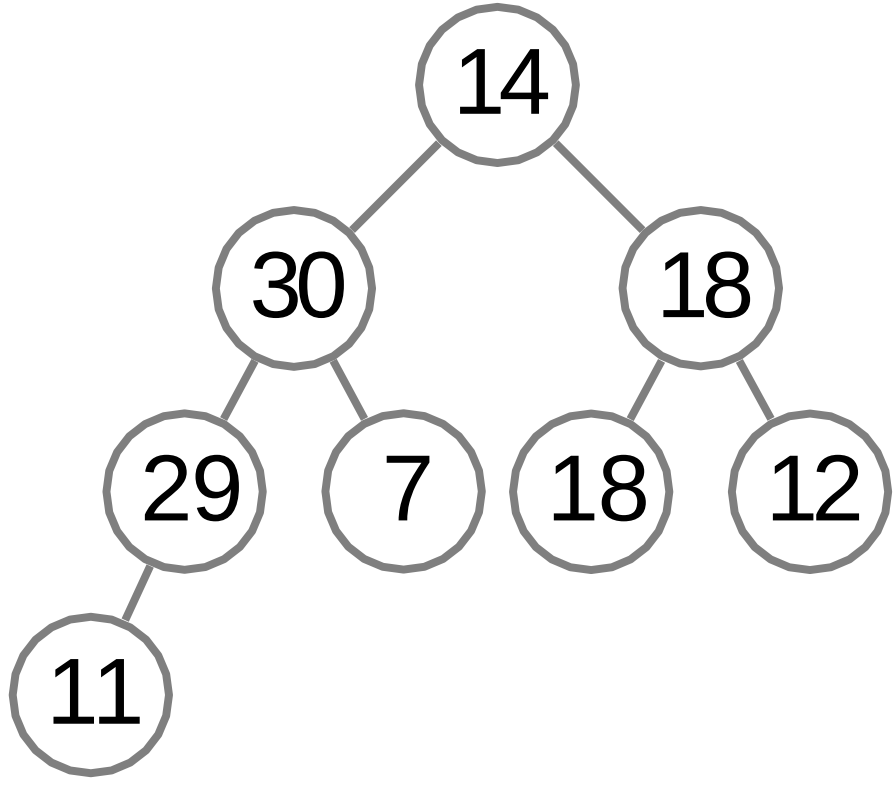
Keeping the Tree Complete

to extract the maximum value, replace the root by the last leaf and let it sift down



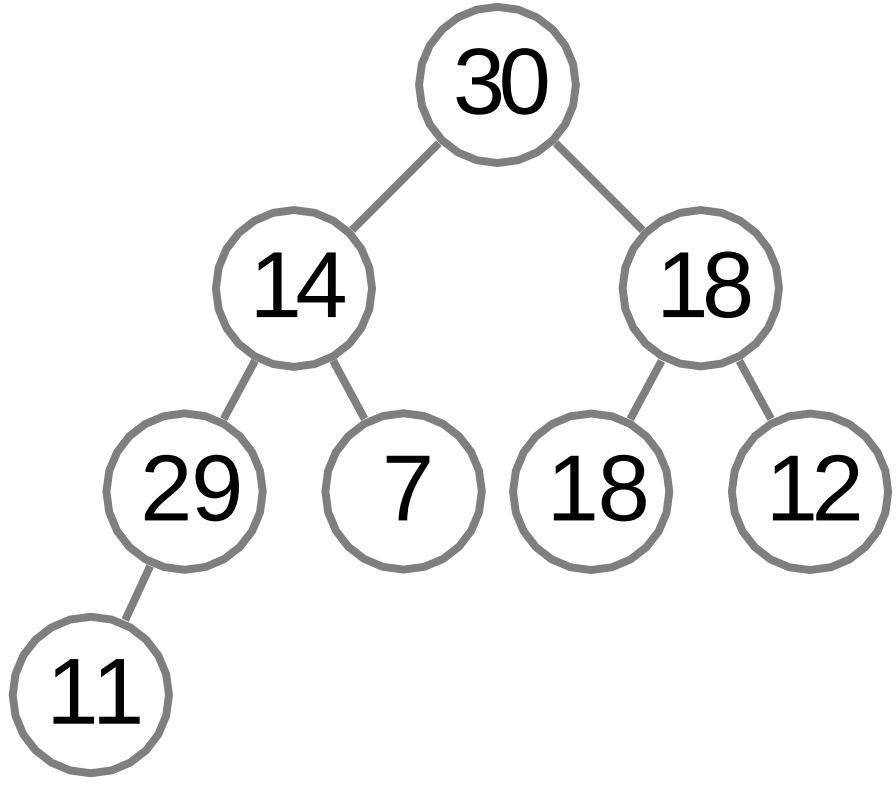
Keeping the Tree Complete

to extract the maximum value, replace the root by the last leaf and let it sift down



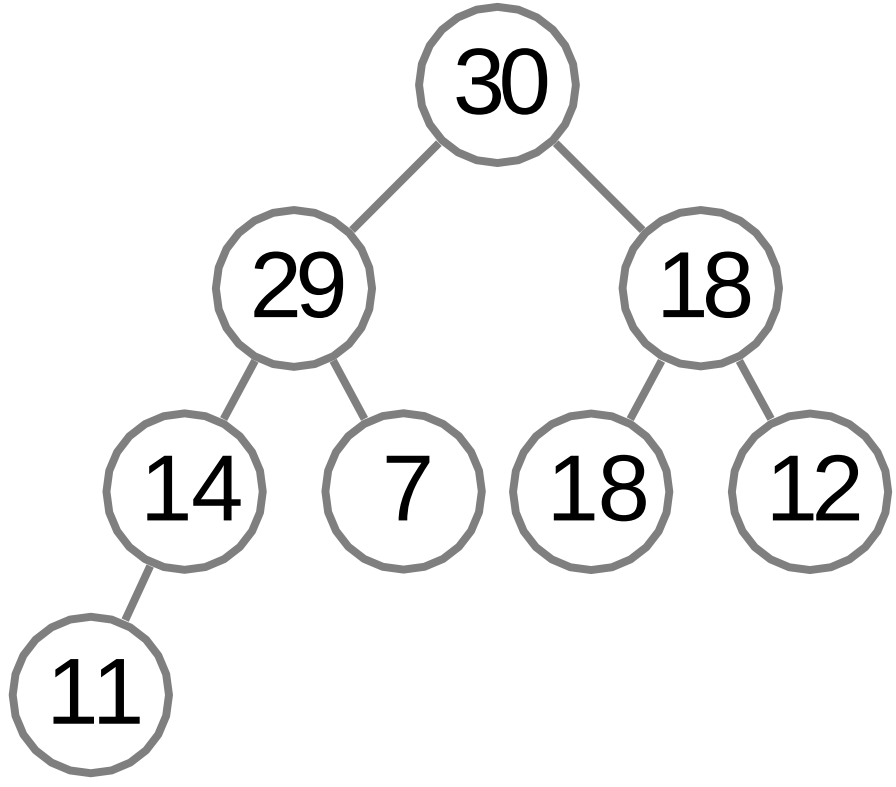
Keeping the Tree Complete

to extract the maximum value, replace the root by the last leaf and let it sift down



Keeping the Tree Complete

to extract the maximum value, replace the root by the last leaf and let it sift down



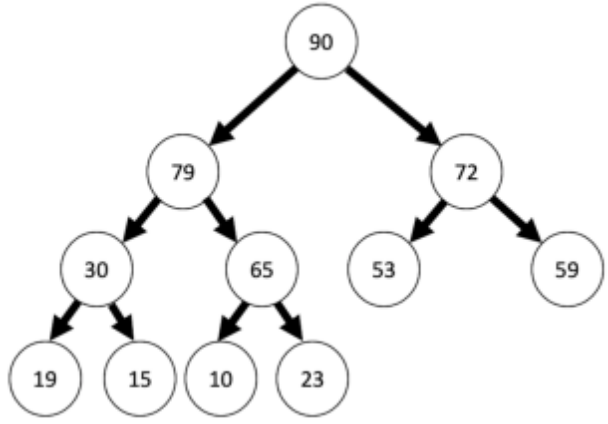
Question 1

(a) Show that the complete binary tree with n nodes has height at most O(log n).
(b)If a complete binary tree has 127 nodes, calculate the maximum possible height of the tree.

Question 2

(a) Consider the following max heap represented as a binary tree:

Show the process of removing the node with value 23 from the heap. Draw all intermediate trees after each step of rebalancing the heap.



(b) Given that a complete binary tree has a height of O(log n), justify why the time complexity of removing a node from a heap and restoring the heap property is O(log n).